

How My Pal JSON works

Deep analysis generated 2026-09-09 · app booted via docker; screenshots are real captures · model claude-sonnet-5

Overview

My Pal JSON is a local, browser-based API client aimed at developers who want a Postman-like workflow without a cloud account or a hosted backend — everything runs on `localhost` via a small Flask app. It lets a user build and send HTTP requests, inspect and pretty-print JSON responses, visualize JSON structure as a tree, and generate model/DTO classes in six languages (C#, Python, JavaScript, C++, Java, Go) from a JSON payload. Its distinguishing feature is that collections are not stored in a proprietary cloud service but pulled from and pushed to a real git repository (Azure DevOps or GitHub) as Postman v2.1 collection files, so a team can keep sharing the same files with people still using actual Postman. A second major feature is a "Chain" tab that lets users compose multi-step request flows, wiring the output of one response (picked directly out of a rendered JSON tree or a header) into `{{variable}}` placeholders consumed by later steps.

The target user is an engineer testing internal or third-party APIs — including localhost services, VPN-gated hosts, or endpoints requiring secrets — who doesn't want request bodies or tokens leaving their machine, and who wants that collection data to live in version control alongside code instead of in a locked-in SaaS tool. The README explicitly frames "secrets never leave your machine" as a value proposition, backed by real implementation choices: auth-header values are marked `isAuth` and are sent as `Bearer <value>` at request time but scrubbed to `{{placeholders}}` before anything is committed back to a repo, and PATs/environment variables are stored only in the browser's IndexedDB, never in the container or in pushed files.

End to end, a user opens the single-page app (`/`), builds a request in the top request bar (method, URL, headers grid with an "IsAuth" column, JSON body textarea), and clicks Send. The browser posts the fully-resolved request description to Flask's `/proxy_request` endpoint, which performs the actual outbound HTTP call server-side (so CORS and TLS/cert trust issues, and container networking quirks around `localhost`, are handled centrally) and returns a normalized envelope (status, headers, timing, content) back to the browser for rendering. The user can then Pretty-format the body, Analyze it into a tree via `/analyze`, or generate typed model classes via `/model`. Collections are managed through a separate "Collections" drawer that talks to `/repo/*` Flask routes, which proxy to Azure DevOps or GitHub REST APIs using a provider abstraction, fetching/writing individual JSON files representing Postman collections or the app's own chain-file format. The Chain tab is pure client-side logic (IndexedDB-backed drafts, a JS request-runner) that also converges on the same `/proxy_request` proxy and the same repo endpoints for persistence.

Environments (named sets of `{{variable}}` values) are a purely client-side, browser-only concept stored in IndexedDB — deliberately never pushed to the repo — used to fill `{{variables}}` in URLs, headers, and bodies both in the single-request tab and in chains.

Architecture

The backend is a minimal Flask app (`web/app.py`) composed of four blueprints with no shared database: `analyze_bp` (`/analyze`, JSON→tree via `anytree`), `model_bp` (`/model`, dispatches to per-language generator modules under `web/generators/`), `proxy_bp` (`/proxy_request`, outbound HTTP proxy using `requests`), and `repo_bp` (`/repo/*`, file-level git operations delegated to `web/providers/`). There is no server-side database or session state — all persistent client state (repos + tokens, environments, chain drafts, settings) lives in the browser's IndexedDB (`web/static/scripts/db.js`), and the Flask process is stateless per request except for an in-process, best-effort "last working host alias" cache in `proxy.py`.

The proxy module is notable: because the app is meant to run inside a container while still reaching services on the developer's own machine, `proxy.py` detects when it's running inside a container (`/.dockerenv` or `/run/.containerenv`), rewrites `localhost` targets to a comma-separated list of `HOST_ALIAS` candidates (defaulting to `host.containers.internal,host.docker.internal`), tries each in order, remembers whichever succeeded

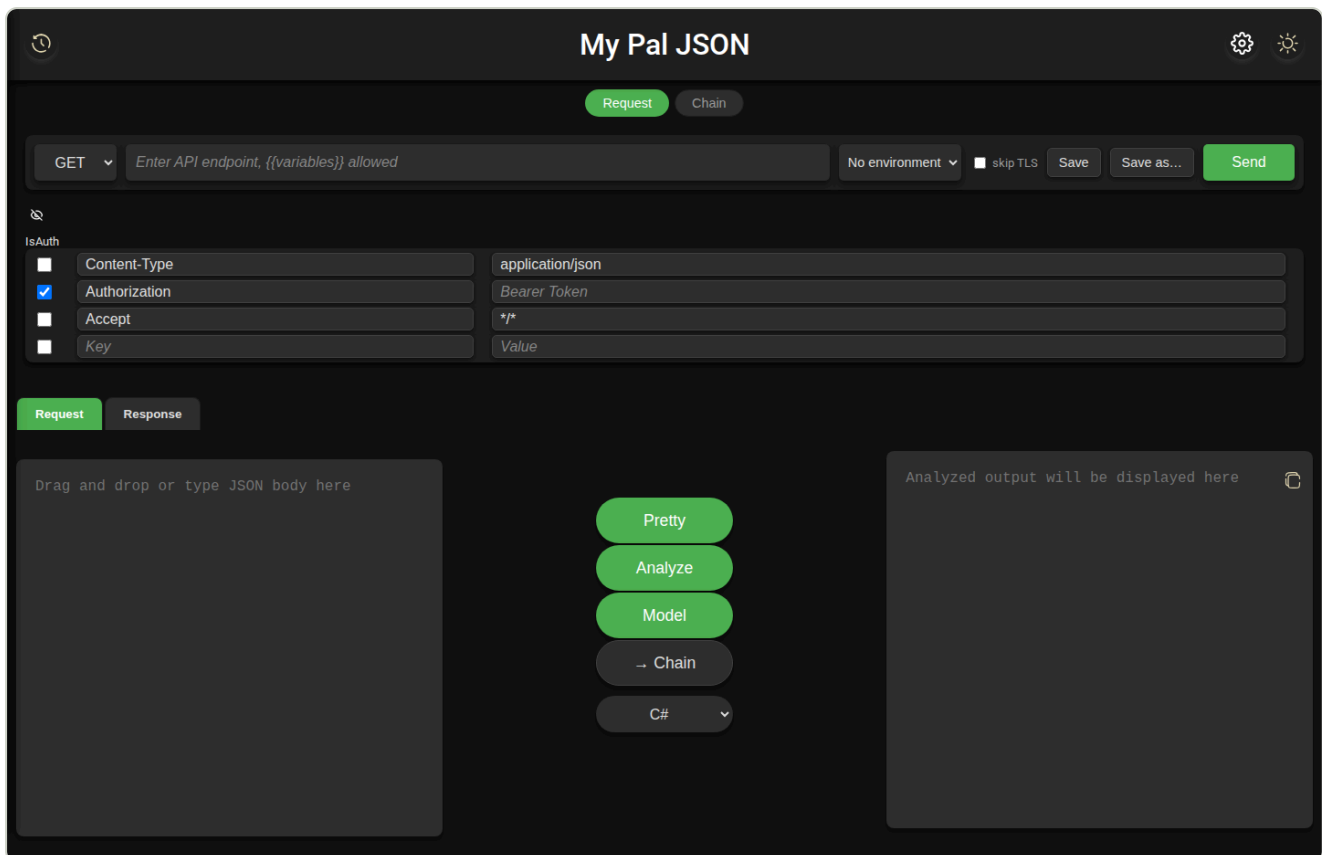
first, and returns actionable error messages (including Windows/WSL-specific guidance) when nothing is reachable. TLS verification is toggleable per-request (``skipTlsCheck`/`verifyTls``), and ``truststore.inject_into_ssl()`` is used at startup so the OS certificate store (including corporate MITM roots) is trusted.

Repo access is abstracted behind ``BaseProvider`` (``web/providers/base.py``) with two concrete implementations, ``AzureDevOpsProvider`` (ADO Git REST API, basic auth with PAT, optimistic-concurrency pushes keyed on branch tip/objectId) and ``GitHubProvider`` (GitHub Contents/Git API, bearer token, blob SHA-based optimistic concurrency). ``web/providers/__init__.py`` parses a pasted repo URL (dev.azure.com, *.visualstudio.com, or github.com patterns) into a ``RepoRef`` and picks the provider. Both providers surface a uniform ``RepoError(status, message)`` that ``repo_bp`` turns into JSON error responses; conflicts (409) surface as "File changed on the server, pull again" to protect against silent overwrites.

The frontend is vanilla ES modules with no build step or framework, loaded by a single Jinja template (``json-analyzer.html``). Key modules: ``send-request.js`` (single-request form state and Send-button flow, calling ``/proxy_request``), ``headers-management.js`/`headers-toggle.js`` (dynamic header grid with auto-growing rows and the `IsAuth` checkbox), ``variables.js`` (``{{var}}`` resolution and JSON-path extraction, shared by the request tab and the chain runner), ``postman.js`` (Postman v2.1 collection JSON $\rightleftharpoons$ internal request-object mapping, preserving unknown fields via ``_raw``), ``collections-store.js`/`collection-browser.js`/`repo-api.js`` (pull/push/tree UI wired to ``/repo/*``), ``environments.js`` and ``settings-panel.js`` (browser-only environment and repo/token management, plus the environment `<select>` in the request bar), and the ``chain/`` subfolder (``chain-store.js`` for IndexedDB drafts and repo push/import of ``*.mypaljson_chain.json``, ``chain-runner.js`` as a pure/async execution engine, ``chain-view.js`` as the DOM controller). ``db.js`` is the single IndexedDB gateway (stores: ``settings``, ``repos``, ``environments``, ``chainDrafts``). Data flow for a normal request is: DOM form $\rightarrow$ ``dataStore.currentRequest()`` $\rightarrow$ ``resolveRequest`` (variable substitution) $\rightarrow$ ``fetch('/proxy_request')`` $\rightarrow$ Flask ``proxy.py`` $\rightarrow$ target API $\rightarrow$ JSON envelope $\rightarrow$ ``renderStatus`/`formatContent`` back into the Response tab. Data flow for collections is: Settings/repo config in IndexedDB $\rightarrow$ ``repo-api.js`` $\rightarrow$ ``/repo/*`` $\rightarrow$ provider $\rightarrow$ ADO/GitHub $\rightarrow$ files cached in ``collections-store.js``'s in-memory ``Map``, parsed by ``postman.js``, rendered as a tree by ``collection-browser.js``, edited in the request form, and written back through the same path with secret-scrubbing before push.

Page: /

The single page of the application; it is the entire My Pal JSON client, providing a request-building/response-inspecting workspace (Request view) and a multi-step request orchestration workspace (Chain view), plus drawers for Collections and Settings.



Walkthrough

A user lands on `/`, served by Flask's `serve\_html` handler which renders `web/templates/json-analyzer.html`. The header shows a Collections icon (left), the title, a Settings gear and a theme toggle (right); a theme is pre-applied before paint from `localStorage`. Below the header, two view tabs — Request and Chain — switch between `#requestView` and `#chainView` via `view-switch.js`, remembering the last-used view in `localStorage`. In the default Request view, the user picks an HTTP method, types a URL (optionally with `{{variables}}`), optionally selects an active environment, ticks "skip TLS" for self-signed certs, and manages headers in a grid where any row can be flagged "IsAuth" so its value is sent as `Bearer <value>` and is never persisted as a literal secret. The user types or drops JSON into the Request textarea, then clicks Send to fire the call through the server-side proxy; the response (status, timing, headers, body) appears under the Response tab. From there Pretty reformats the body locally, Analyze posts it to `/analyze` for a tree view, and Model posts it to `/model` (with a chosen target language) to generate class/struct definitions, all shown in the read-only output box with a Copy button. "→ Chain" sends the current request into the open chain as a new step. Separately, the Collections drawer (opened by the top-left icon) lets the user pick a repository/branch, Pull collection and chain files from it, browse/edit/save requests, and Push changes back with secret-scrubbing and conflict detection. The Settings gear opens a modal for configuring repositories/tokens and environments. Switching to the Chain tab reveals a chain picker, run controls, and a vertical list of step cards that can be run individually or as a whole, with drag-to-reorder and value-picking from a rendered response tree.

Controls

CONTROL	WHAT IT DOES
Request / Chain tabs (`#viewRequestTab` / `#viewChainTab`)	Toggles between `#requestView` and `#chainView` sections; handled by `showView()` in `web/static/scripts/view-switch.js`, persists the choice to `localStorage`.
HTTP method select (`#httpMethod`)	Sets the method used when building the outgoing request; read by `dataStore.currentRequest()` in `send-request.js` and sent to `/proxy_request`.
API URL input (`#apiUrl`, "Enter API endpoint, {{variables}} allowed")	Holds the request URL, including `{{variable}}` placeholders resolved by `resolveRequest()` in `variables.js` against the active environment before sending.
Environment select (`#envSelect`, "No environment")	Chooses the active environment (from IndexedDB `environments` store via `environments.js`) used to fill `{{variables}}`; managed/rendered by `settings-panel.js`.
skip TLS checkbox (`#skipTlsCheck`)	Passed as `verifyTls` in the request body to `/proxy_request`; server disables `urllib3` TLS warnings and sets `requests`' `verify=False` for that call (`proxy.py`'s `proxy_request` handler).
Save button (`#saveRequestBtn`, "Save")	Writes the current form back into the currently loaded collection request (Ctrl+S shortcut), updating the in-memory collection entry via `collections-store.js`/`postman.js` (`requestToItem`), marking the file dirty for later Push.
Save as... button (`#saveAsBtn`)	Saves the current form as a new request elsewhere in a collection (via the collection browser's add/duplicate logic in `postman.js`/`collections-store.js`).
Send button (`#sendBtn`, class `send-button`)	Triggers `sendRequest()` in `send-request.js`, which POSTs the resolved request to Flask's `/proxy_request` (`proxy.py: proxy_request`), then renders status/headers/body into the Response tab via `renderStatus`/`formatContent`.
Toggle Headers button	Shows/hides the headers grid and its IsAuth label; handled in `headers-toggle.js` by toggling `display` and swapping the eye/eye-slash icon.

```
(`#toggleH  
eadersBtn`  
)
```

**Auth
checkbox
per header
row
(unlabeled,
title "Auth
header:
sent as
'Bearer
<value>'
and never
written to
the repo as
a literal")**

Marks a header row as auth; ``collectHeaders()`` in ``send-request.js`` prefixes its value with ``Bearer`` when sending, and ``postman.js`` chain push logic (``findLiteralSecrets``) blocks pushing literal values for such headers.

**Header Key
input
(`.header-
key-input`,
placeholde
r "Key")**

Header name field in the dynamically managed grid (``headers-management.js``: `createHeaderRow``); grid auto-adds/removes rows as you type (``addInputListeners``).

**Header
Value input
(`.header-
value`,
placeholde
r "Value" or
"Bearer
Token" for
Authorizati
on)**

Header value field, read by ``readHeaders()`` and written by ``collectHeaders()`` for sending.

**Request /
Response
tabs
(`#request
Tab` /
`#response
Tab`)**

Switches the textarea (``#inputJson``) between the outgoing request body and the last response body, per ``tab-switch.js``, using ``dataStore.requestData``/``responseData``.

**Request
body
textarea
(`#inputJso
n`, "Drag
and drop
or type
JSON body
here")**

Holds the JSON to send/analyze/model; supports drag-and-drop file loading via ``input-drop.js``, which reads a dropped ``json``/text file into the textarea.

Pretty button (`#prettyBtn`)	Client-side only; sanitizes and re-serializes the textarea JSON with 4-space indentation into `#outputBox` (`make-pretty.js`), showing an "Invalid JSON" message on failure.
Analyze button (`#analyzeBtn`)	POSTs the parsed JSON body to Flask `/analyze` (`analyze.py: process_json`), which builds an `anytree` structure and returns a rendered tree string shown in `#outputBox` (`validate-json.js`).
Model button (`#modelBtn`)	POSTs the parsed JSON body to Flask `/model?language=...` (`model.py: generate_classes`), which dispatches to the matching generator in `web/generators/` and returns concatenated class/struct source shown in `#outputBox` (`model-json.js`).
→ Chain button (`#sendToChainBtn`)	Adds the current Request-tab form as a new step in the currently open chain (wired through `chain-view.js`'s step-adding logic using `newStep()` from `chain-store.js`).
Language select (`#languageSelect`)	Chooses the target language (csharp/python/javascript/cplusplus/java/go) sent as the `language` query param to `/model`.
Copy button (`#copyBtn`)	Copies `#outputBox` contents to the clipboard via the Clipboard API, flashing green/red on success/failure (`copy-output.js`).
Repository select (`#repoSelect`, "No repositories - open Settings")	Lists saved repos (IndexedDB `repos` store) and sets the active repo via `store.setActiveRepo()`/`store.getRepo()` in `collections-store.js`, rendered by `collection-browser.js: renderRepoControls`.
Branch select (`#branchSelect`)	Chooses the active branch for the selected repo; persisted via `store.setBranch()`, which clears cached files and re-emits a repo-changed event.
Pull button (`#pullBtn`, "Pull")	Calls `store.pull()` in `collections-store.js`, which lists the repo root (and one folder deep) via `repoApi.getTree`, fetches every `*.postman_collection.json`/`*.mypaljson_chain.json` file via `repoApi.getFile`, and populates the in-memory file map/tree (`collection-browser.js: doPull`).
Push button (`#pushBtn`)	Commits changed/dirty collection files back to the repo (one file per change) via `collections-store.js`/`repo-api.js: putFile`/`deleteFile`, scrubbing secret-looking values into `{ {placeholders} }` first and surfacing 409 "changed on the server, pull again" conflicts.
"+" button	Creates a new collection file at the repo root/selected folder (`store.createCollectionFile()` in `collections-store.js`), shown immediately in the collection tree as a dirty/unpushed file.
⚙️ / Settings icon button	Opens the Settings modal (`settings-panel.js: openSettings()`), defaulting to the Repositories pane, for managing repo URLs/PATs and environments.

**(`#settings
Btn`)**

"Open Settings" button (inside empty-tree state)	Same as the gear icon; calls `openSettings("repos")` when no repository is configured (`collection-browser.js: renderTree`s `#treeOpenSettings` handler).
---	---
